

Assignment 2 – Part A Object Recognition Implementation

11376/8890 – Computer Vision and Image Analysis (CVIA)

Context

This assignment focuses on the fundamental computer vision tasks of object detection and recognition. The goal is to automatically identify and classify objects within images. In this assignment, you will develop and evaluate methods to detect and recognise bird species from images, gaining practical experience with both classical and deep learning approaches.

Dataset



Figure 1. Example images: Parakeet Auklet (left), Green Kingfisher (centre), and Horned Puffin (right)

We will use the Caltech-UCSD-Birds ([CUB-200-2011](#)) dataset. It is a widely used benchmark for fine-grained image classification tasks. It contains 11,788 images across 200 bird species, with approximately 40–60 images per class. Each image is annotated with class labels and bounding boxes that indicate the location of the bird within the image. The dataset presents significant challenges due to variations in pose, lighting, background clutter, and subtle inter-class differences, making it well suited for evaluating both classical and deep learning-based object detection and recognition methods.

- 11,788 images
- 200 bird species (approximately 40–60 images per class)
- Available on [Canvas -> Modules -> Assignment 2 Information](#)
- Class labels and bounding boxes are provided for each image.
- The `images` folder contains all the images, with a subfolder for each of the 200 classes.
- The `images.txt` file contains a list of the individual image file names including the subfolder path.
- The `classes.txt` file contains the class names (bird species). The dataset is 'benign' in the sense that each image contains only one bird.

- The `image_class_labels.txt` file contains the ground truth class labels (bird species) for each image.
- The `bounding_boxes.txt` file contains the (x, y, width, height) of a single bounding box for each image, describing the image area that contains the bird.
- The `train_test_split.txt` file explicitly defines the training and testing split. The first column is image index, and the second column indicates whether the image is used for training or testing: 1-training and 0-testing.
- More information about the structure of each of these files can be found in the `README.txt` file.

Data Split

We will use the training and testing split specified in the `train_test_split.txt` file for our experiments. Within the training set, a further split can be performed to create a validation set. This validation set is used for hyperparameter tuning and for monitoring model performance during training (e.g., early stopping and model selection).

Task:

You are required to implement and evaluate two types of approaches:

1. Classical Approach:

A handcrafted feature-based method combined with a traditional machine learning classifier.

Examples:

- HOG features + Support Vector Machine (SVM)
- Other feature + classifier combinations

2. Deep Learning Approach:

A method where both features and classifier are learned automatically.

Examples:

- Convolutional Neural Networks (CNNs)
- Others or more advanced models
- Pretrained models with transfer learning

The choice of method/approach is up to you, as are the specific parameter settings. For example, you may decide whether to resize the images first (strongly recommended), select the number of layers in your deep learning model, and choose an appropriate network architecture. All design decisions must be clearly documented in your Python code and thoroughly described in your report.

Experimental Design:

Both approaches must be evaluated under two input settings:

1. The entire images are used as input.
2. Only the image area marked by the bounding box information is used as input.

Consequently, you will need to perform **four experiments**:

Input Type	Classical Method	Deep Learning Method
Whole Image	Experiment 1	Experiment 2
Bounding Box Region Only	Experiment 3	Experiment 4

Evaluation:

Performance should be evaluated on the test set only, using:

- Overall classification accuracy (all classes)
- Class-wise accuracy (accuracy for each class)

Submission Requirements

- **A PDF report** – single line spacing, font size 10, maximum 5 pages. It should include:
 - Method Description: Explain each selected algorithm and any parameter choices. [10 marks per method (class vs deep learning) = 20 marks]
 - Results and Comparison: Present and discuss the results of the performance evaluation of the four experiments. [10 marks for each of the four experiments = 40 marks]
 - Discussion and lessons learnt: Strengths, weaknesses, potential improvements. [5 marks]
- **Python code** – everything in a single Zip file, well-documented with instructions to run the code (if necessary)
 - Good programming principles and practice: Demonstrated the use of good programming principles such as clear program structures, good use of comments, use of functions (where appropriate) etc. [10 marks]

Total: 75 marks

Bonus (5 marks)

Perform 5-fold cross-validation on any one of the four experiments (only one is required), and report the average accuracy across folds.

In 5-fold cross-validation, data for each class is divided into five equal parts. The experiment is then run five times, with different parts used for training, validation, and testing for each run. For example, you can use the following split:

- Run 1: Parts 1-3 for training, Part 4 for validation, Part 5 for testing
- Run 2: Parts 2-4 for training, Part 5 for validation, Part 1 for testing
- Run 3: Parts 3-5 for training, Part 1 for validation, Part 2 for testing
- Run 4: Parts 4, 5, and 1 for training, Part 2 for validation, Part 3 for testing
- Run 5: Parts 5, 1, and 2 for training, Part 3 for validation, Part 4 for testing

Notes

- Tune parameters carefully using a validation set.
- Recommended libraries: OpenCV, scikit-image, NumPy, etc.
- Refer to Week 11 Lab and Week 10 Lab materials for implementation guidance.